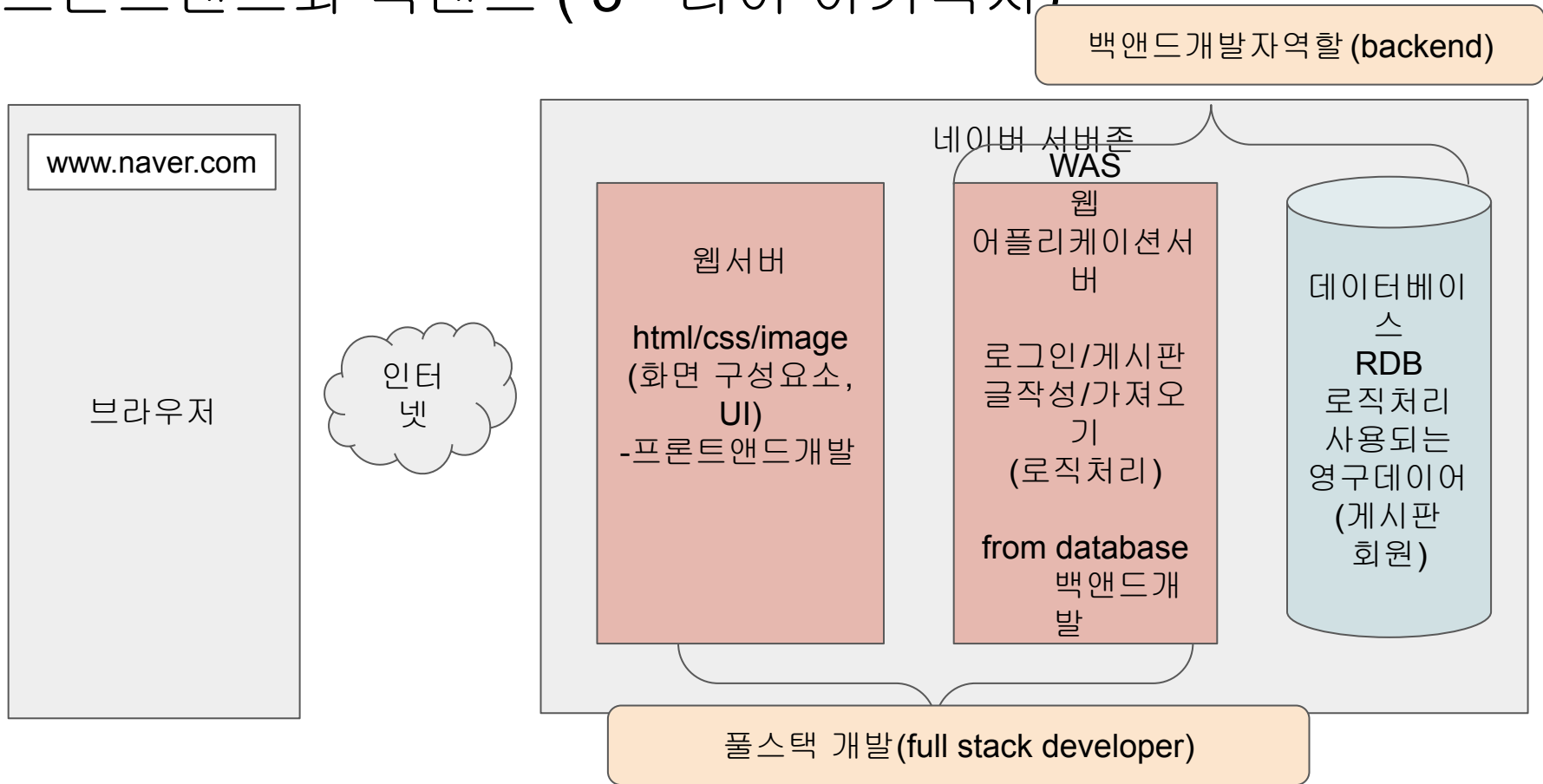


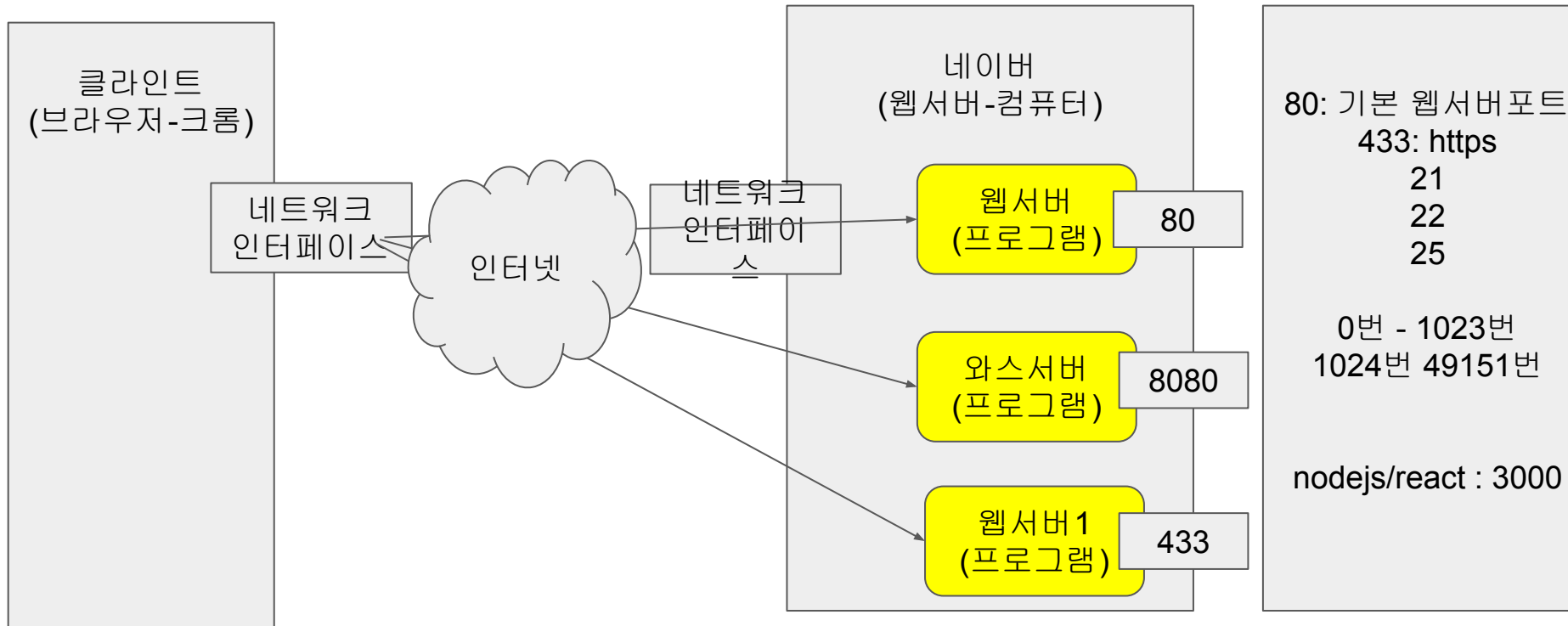
강서 클라우드기반 AI활용 리액트 개발자 양성과정

2026.06.25

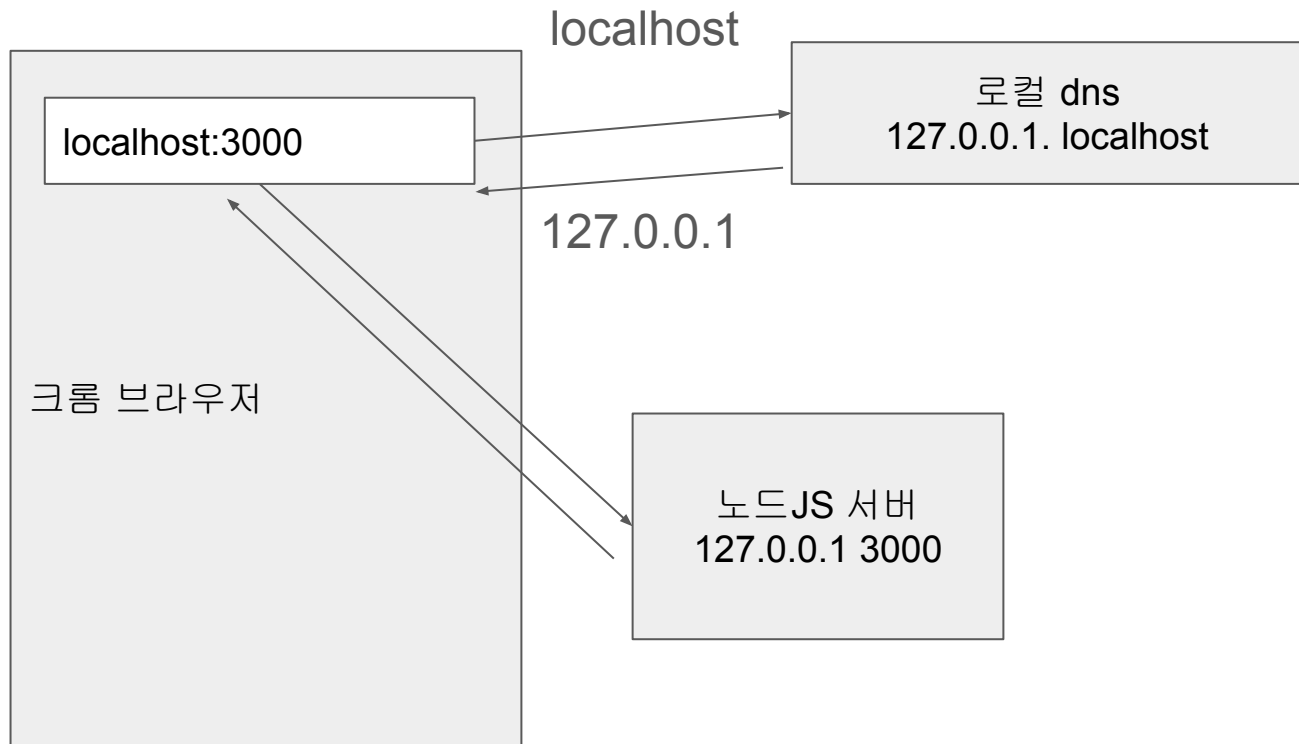
프론트엔드와 백엔드 (3 - 티어 아키텍처)



포트 및 인터넷 동작 방식



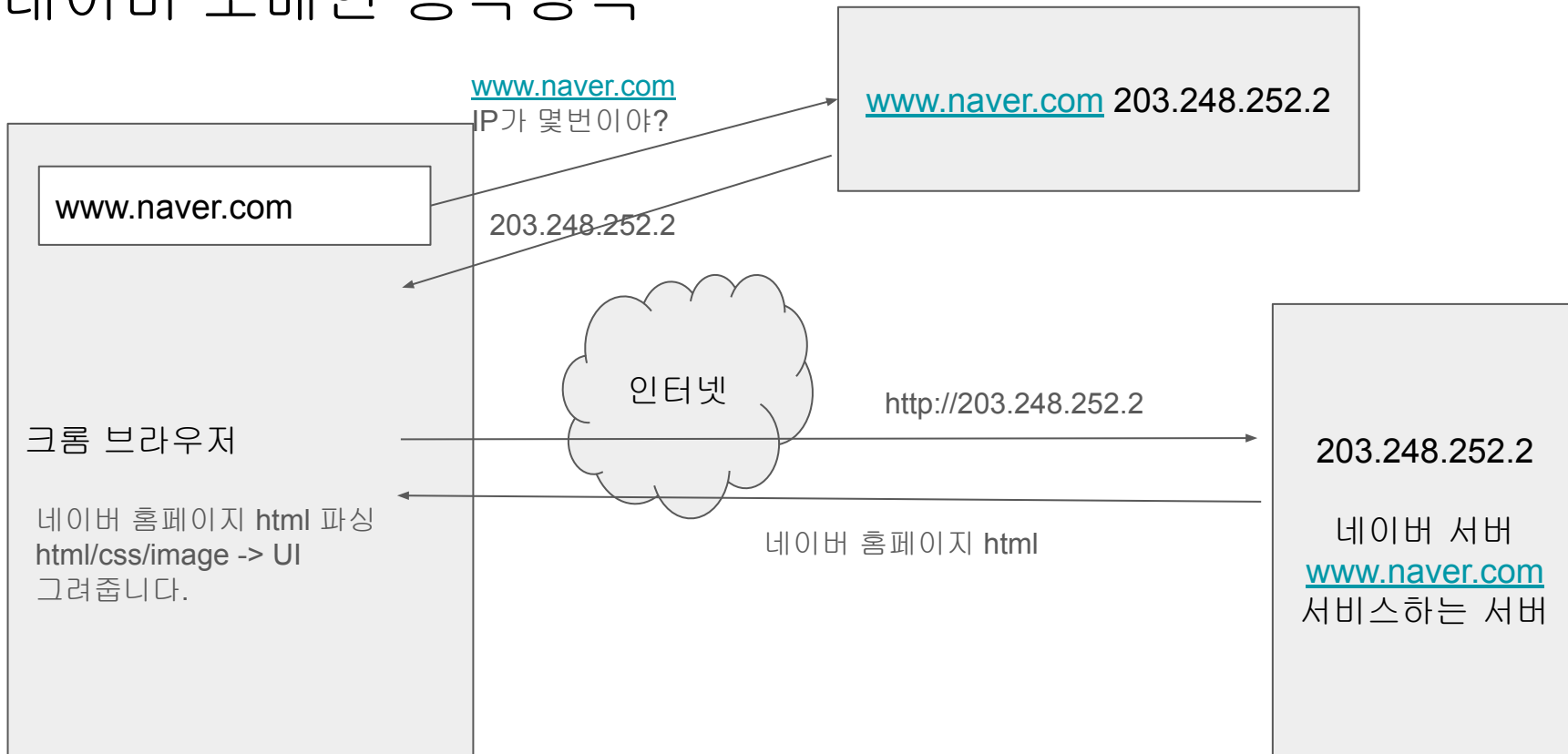
도메인 동작방식



127.0.0.1 IP 주소

localhost
도메인이름

네이버 도메인 동작방식

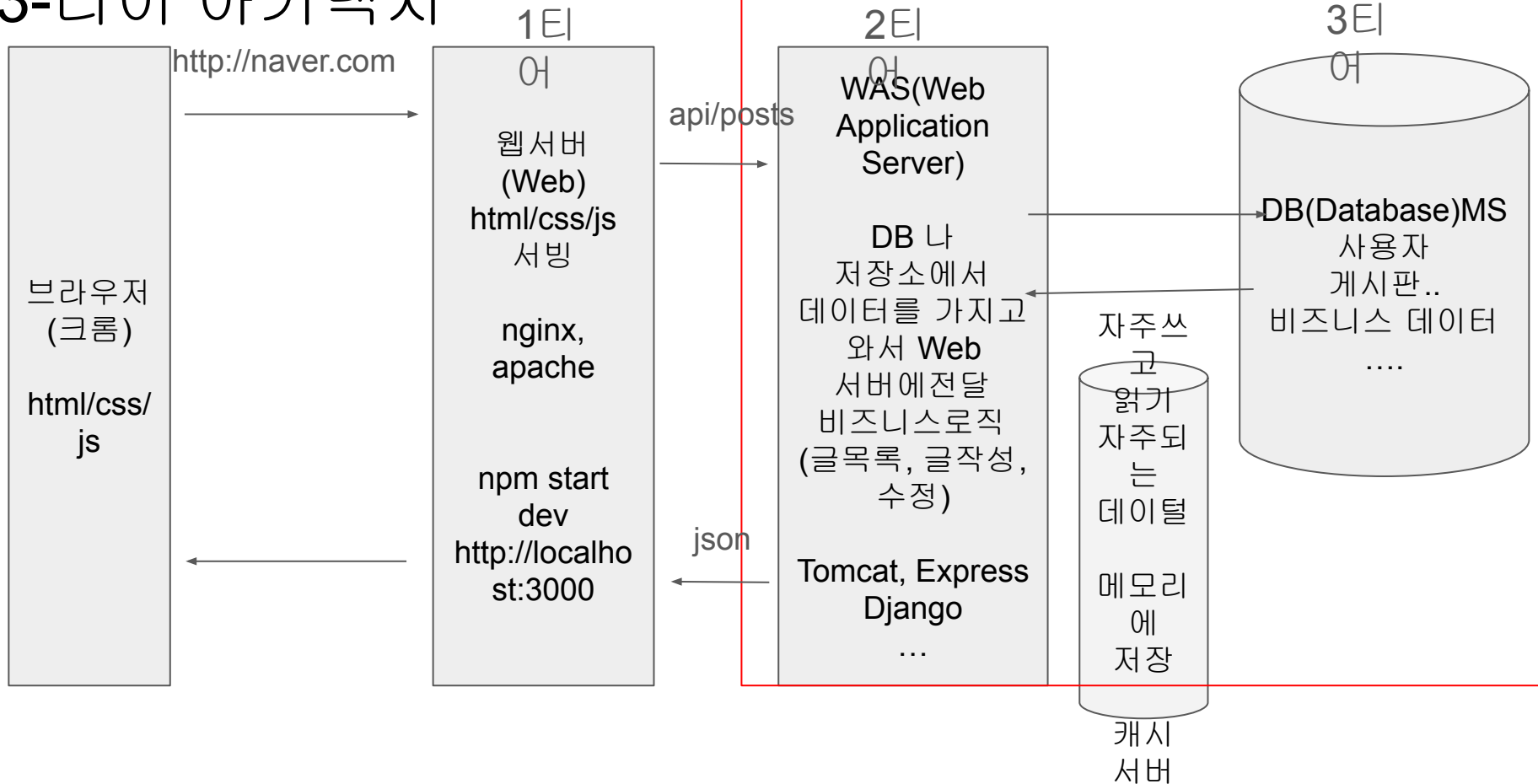


관계형데이터베이스

테이블로 데이터를 저장하는데

엑셀 과 유사한 형식으로 데이터를 저장

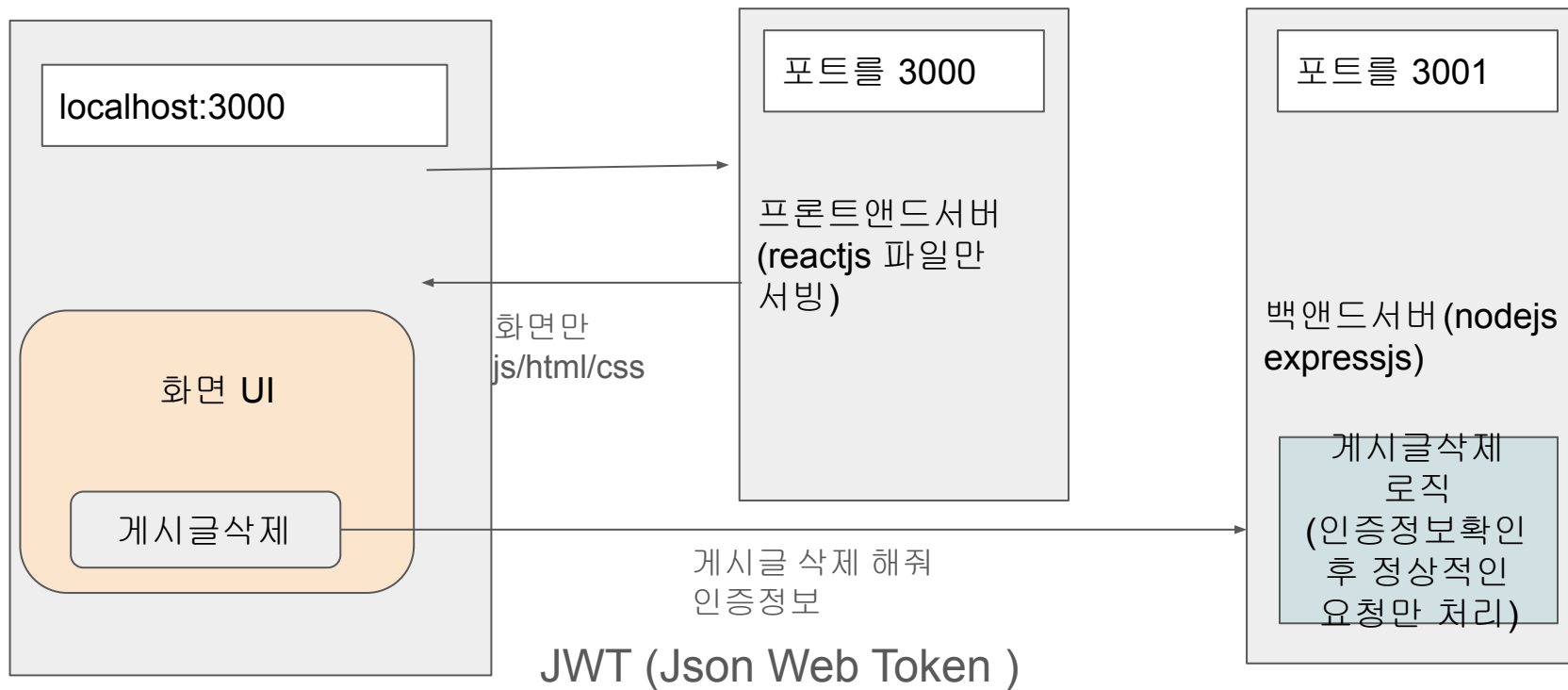
3-티어 아키텍처



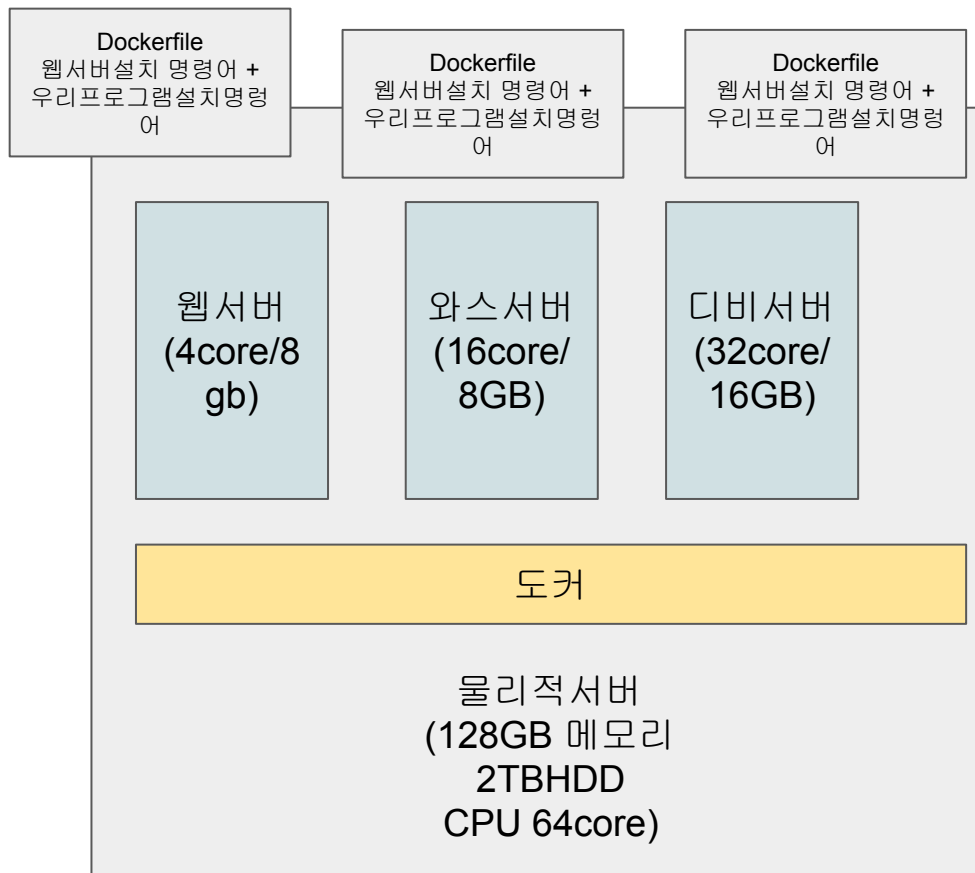
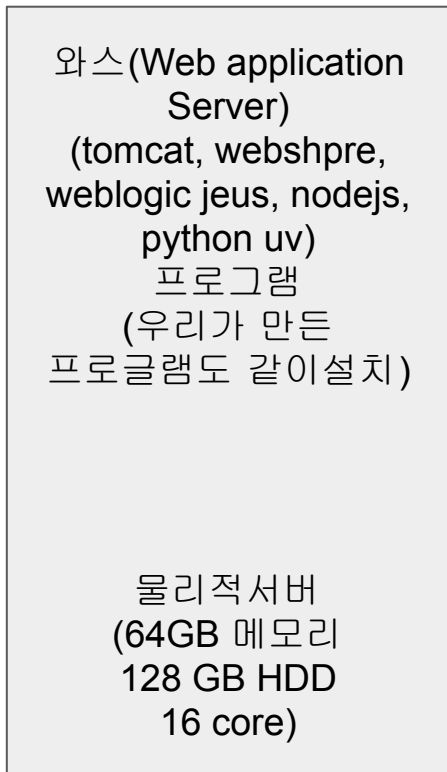
저장속도 - 컴퓨터에 저장되는

1. 캐시메모리
2. 메모리 (64GB)
3. SSD (128 GB) -> 메모리
4. HDD (1TB). -> 원반이 물리적으로 빙글빙글 돌면서 데이터를
5. Tape (백업) -> 카세트 테이프

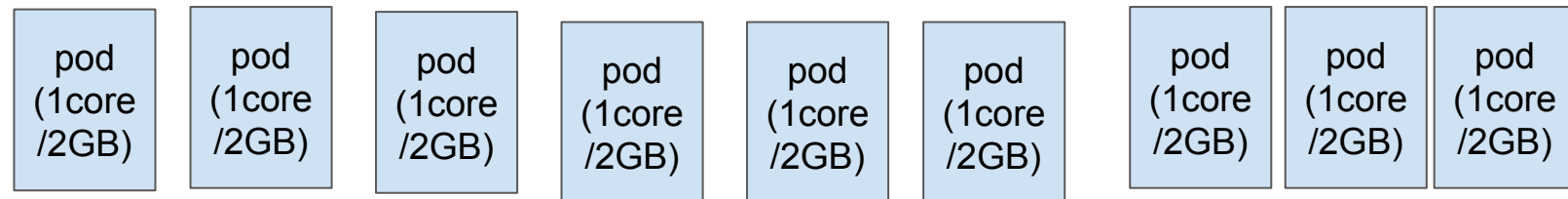
인증(JWT)



도커



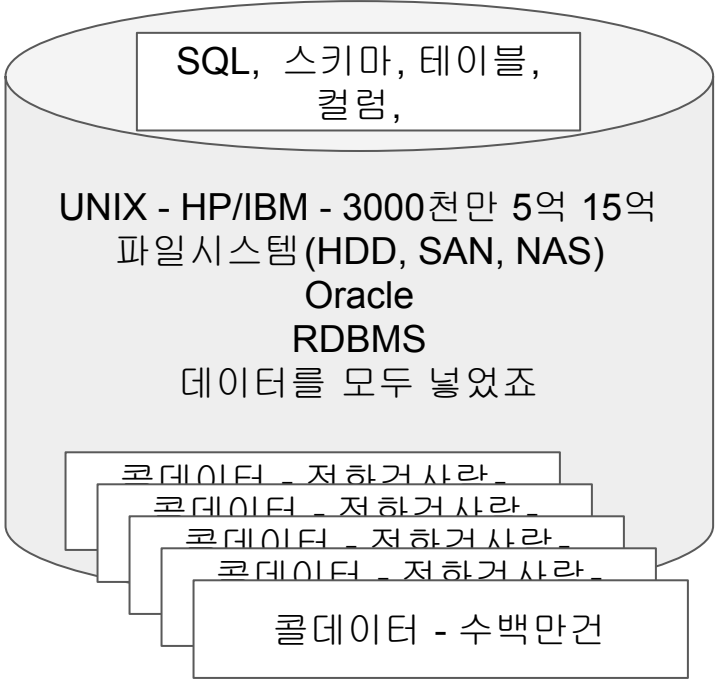
쿠버네티스



쿠버네티스 (하나의 컴퓨터 처럼 과리 해줘요)

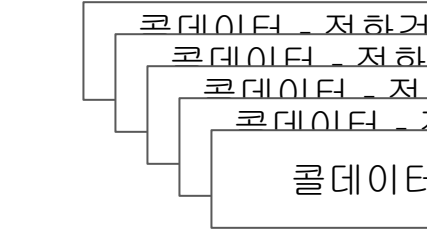


하둡 - NoSQL



BI(Bisness Intelligent) 시스템

Spark 와 같이 전용 쿼리 엔진 (프로그램)



Hadoop(일종의 가상 파일)

싼
리눅스머신
(500만원)

싼
리눅스
(500만

NoSQL
HBASE
MongoDB -
JSON(Javascript object
Notation)
(RAG VectorStore)
DynamoDB

몽고디비

몽고디비

싼 머신1

샤드1

샤드2

샤드3

싼 머신2

샤드1

샤드2

샤드3

싼 머신3

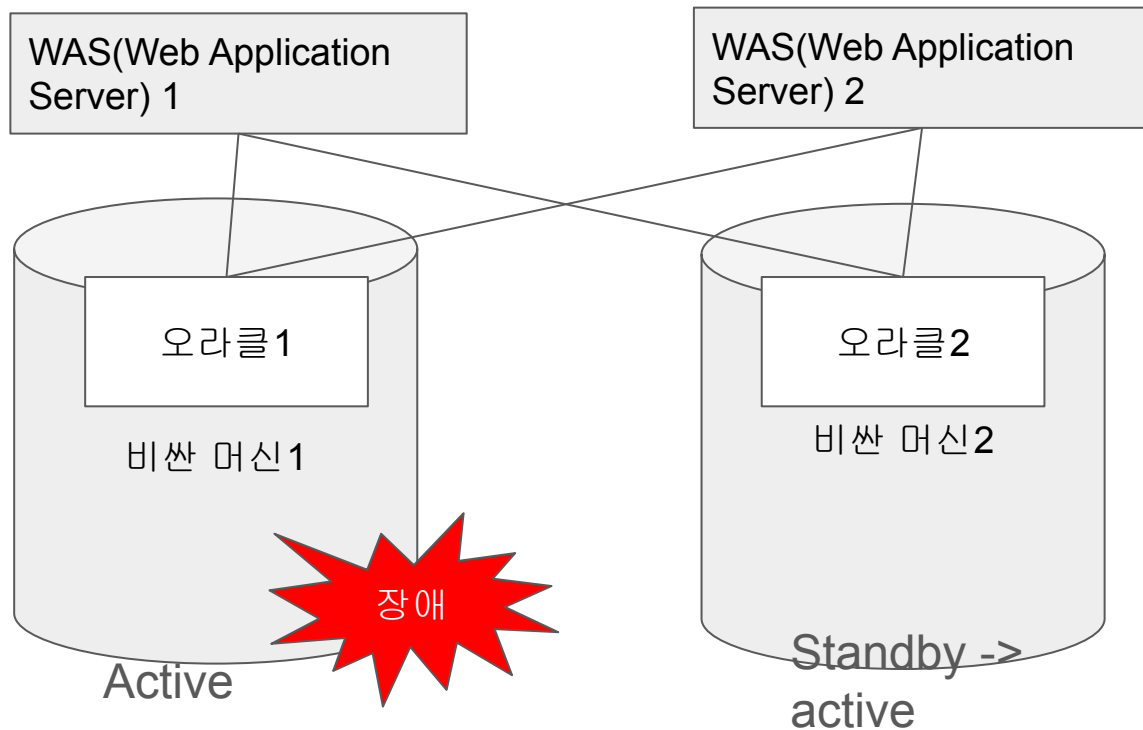
샤드1

샤드2

샤드3

장애발생

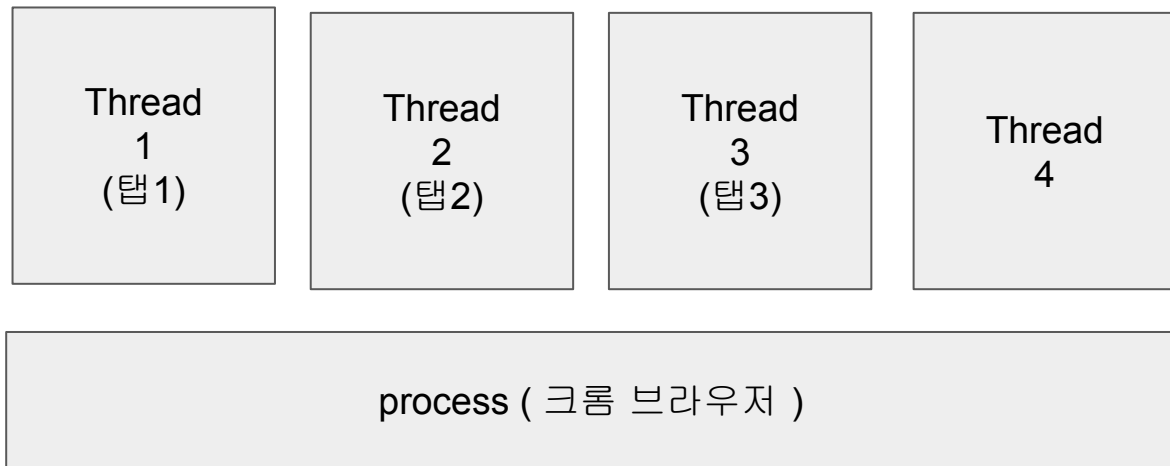
RDB



실시간연
결
골든게이
터

프로세스와 쓰레드

1. 엑셀 프로그램 실행 시키면 엑셀프롬 -> 프로세스 (process)



동기와 비동기

console.log(1)

console.log(2)

console.log(3)

1

2

3

순차실행 :
동기

Thread
1
(탭1)

const data = fetch(["https://example.com/users"](https://example.com/users)) :
13초 걸렸어요 -> 스레드1

console.log(data) : 13걸치

// 다음 작업 화면에서 UI 그려주는 작업 13초 이상
->스레드 1

// 버튼을 눌러도 반응이없고

// 스크롤, 애니메이션 멈춤

// 페이지 동작하지 않는 것처럼 > 싱글
스레드라서 발생문제

자바스크립트 비동기

```
console.log("1. 주문 넣음")
```

```
fetch("http://example.com/users").then(function(response){ console.log("3") })
```

```
console.log("2 다른일 계속")
```

```
// 비동기 1 -> 2 -> 3(30초가 걸리든 13초가 걸리든)
```

```
// 동기 : 1 > 3 > 2
```

이벤트 루프

- 호출 스택 : 지금 당장 실행 중인 코드
- **Web API** : 브라우저가 대신 처리하는 일 (타이머, 네트워크 작업)
- 태스크 큐 : 끝났으니까 이 함수를 실행 콜백
- 이벤트 루프 : 호출 스택이 비면 큐에서 콜백을 하나꺼서 실행

Promise

동기(블로킹)

1. 커피를 주문
2. 커피를 제작해 -> 블로킹
(기다리고있는거죠)
3. 커피를 전달해 ->
무한대기

비동기(블로킹)

1. 커피를 주문
2. 진동벨을 전달해
3. 커피를 제작해 -> 블로킹
4. 커피가 다되면
5. 진동벨을 울려
6. 커피를 전달해

Promise

- 진동벨로 생각하시면될것 같고
- 지금은 없지만 나중에 생길 값을 담는 상자
- 상자를 받는 시점에는 값이 없어요.. 그래서 **Promise** 는 값이 (커피) 아니다
- 값이 도착할 거라는 약속(진동벨)만 가지고 있고

Promise 커피가 나온다는 약속만 가지고

1. 주문하고
2. 진동벨을 받고
3. 자리에 와서 다른일(책을 본다던가..)
4. 벨이 울리면 수령

즉 커피는 뒤에서 누군가가 열심히 만들고 있었어요(그 누군가는 아까 이벤트루프, libuv)

Promise 에는 3가지 상태

- `pending` : 대기중, 진동벨을 받았지만, 아직 울리지 않음
- `fulfilled` : 이행(성공), 벨이 울림 , 커피를 받으러감 `.then(()=>)`
- `rejected` : 거부, 커피를 만들다가 원두가 다 떨어져서 못만들 환불 받음 `.catch()`

코드상으로 본다

```
const coffee = orderCoffer(); // coffee 라는 애는 진동벨(Promise 객체)
```

```
coffee
```

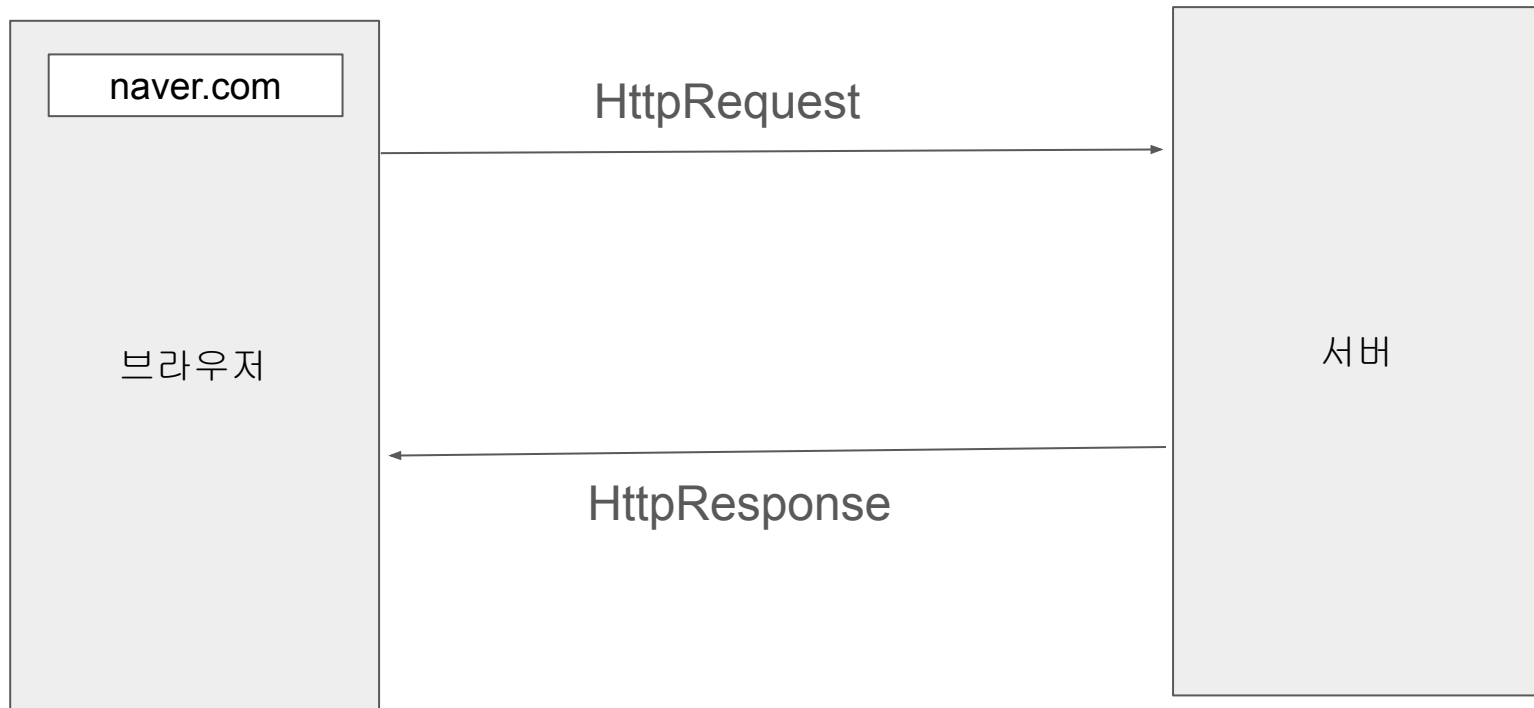
```
.then(cup => console.log("커피를 받음"))
```

```
.catch(err => console.log("재료 떨어져서 환불 받음" )
```

async/ await

```
async function morning() {  
    // 벨이 끝날 때 까지 이 줄에서 기다린다.  
    const cup = await orderCoffee(); // promise  
    console.log(cup)  
}
```


HTTP



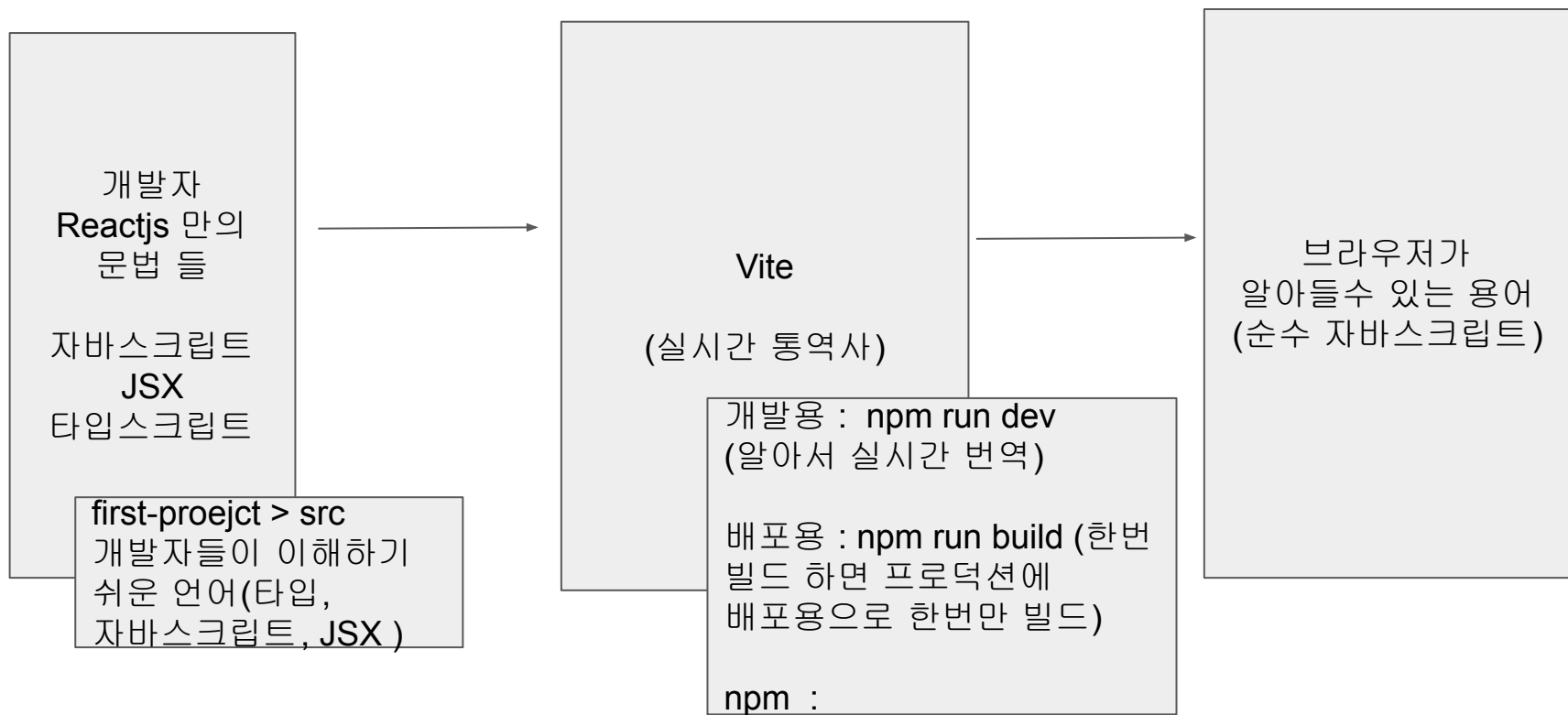
HttpRequest

구성요소	설명	예시
헤더정보	메타데이터를 담고 있어요(인증정보 등등)	인증
주소	URL	www.naver.com
파마리터	주소 뒤에 서버에 전달하는 문자열	https://www.82cook.com/entiz/enti.php?bn=15
바디 정보		
HTTP 메서드	GET, POST, PUT, FETCH, DELETE	

HttpResponse

구성요소	설명	
응답코드	200 (성공), 404(리소스가없), 500(서버에러)	
응답본문	실제 데이터 담겨져 있는 텍스트	
응답헤더	응답 본문의 데이터 형태 (mime-type : application/json, text/html ...)	

Vite 는 왜 필요할까요?



서버 배포 node 모듈 배포 (프론트 앤드)

web
(reactjs)

웹서버 밑에 정적파일 (js
묶어서) 서빙
npm run build
/dist 빌드된 파일이 있으면
rsync, scp

was
(nodejs, expressjs,
nestjs)
pm2 서버구동 런타임

DB

toggle todo 설명 - toggleTodo(2)

```
todos.map((todo) => todo.id === id ? {...todo, done: !todo.done} : todo)
```

todos =

newTodos

[

{id: 1, text:"오늘저녁끓기", done:false},

{id: 2, text:"오늘달리기", done:false},

{id: 3, text:"오늘듀오링고", done:false},

]

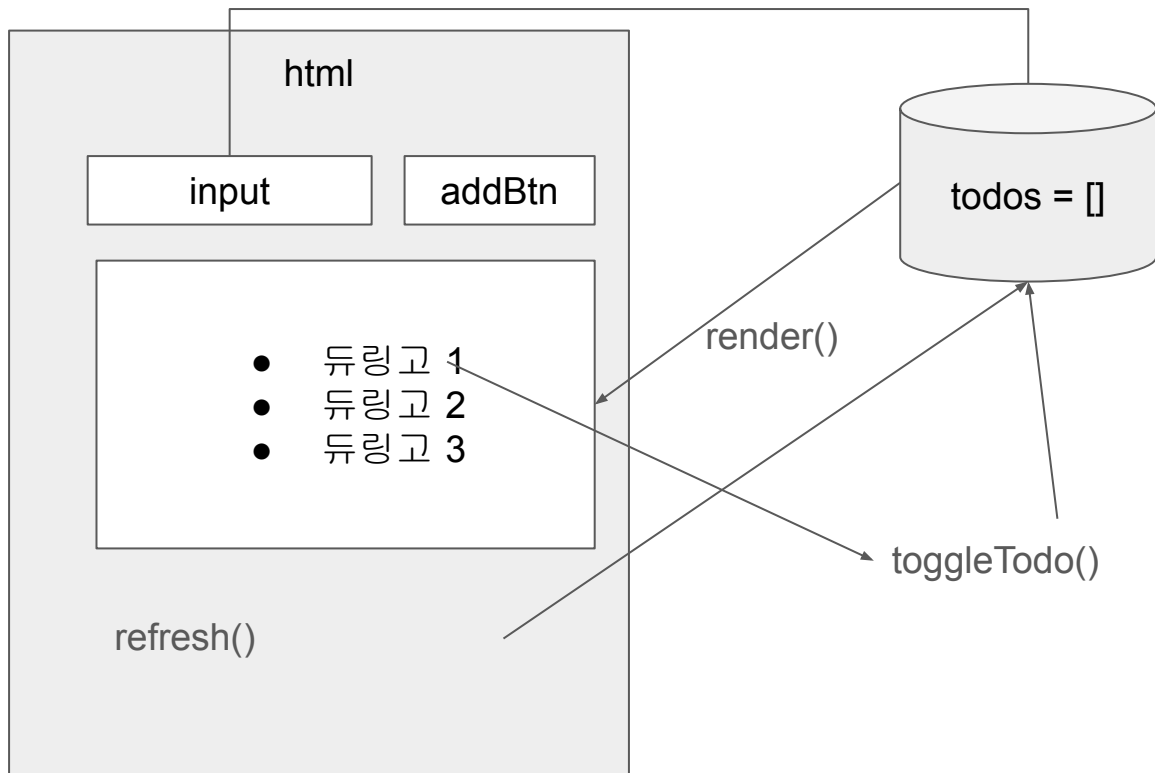
map(1) : 1 === 2 ? 참일경우로직:
거짓일경우로직
newTodos.push(todo)

map(2) : 2 === 2 ?
{id: 2, text:"오늘달리기",done: true }
newTodos.push(todo)

map(3) : 1 === 3 ? 참일경우로직: todo
newTodos.push(todo)

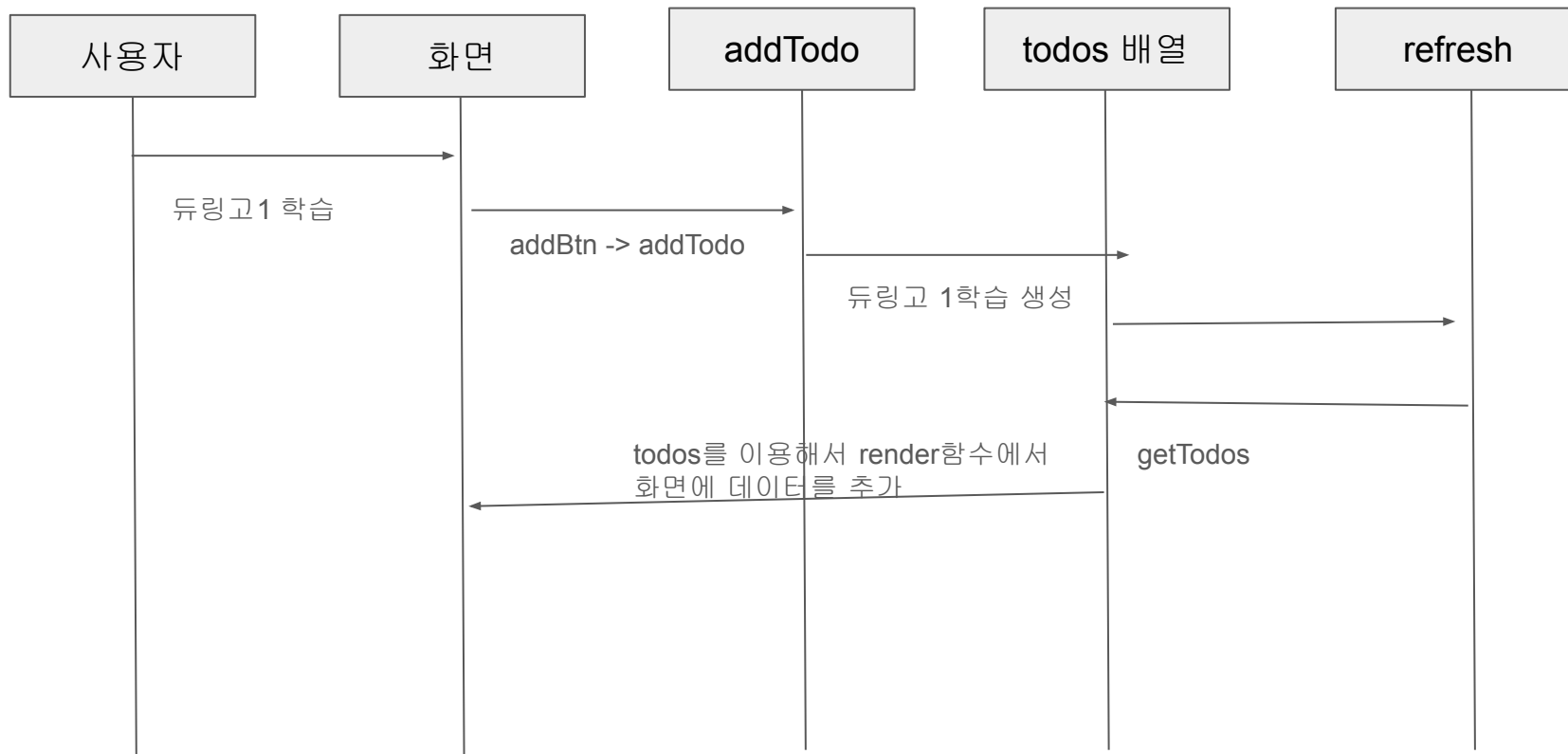
Todo-전체 그림

```
addBtn.addEventListener("click", () => {})
```

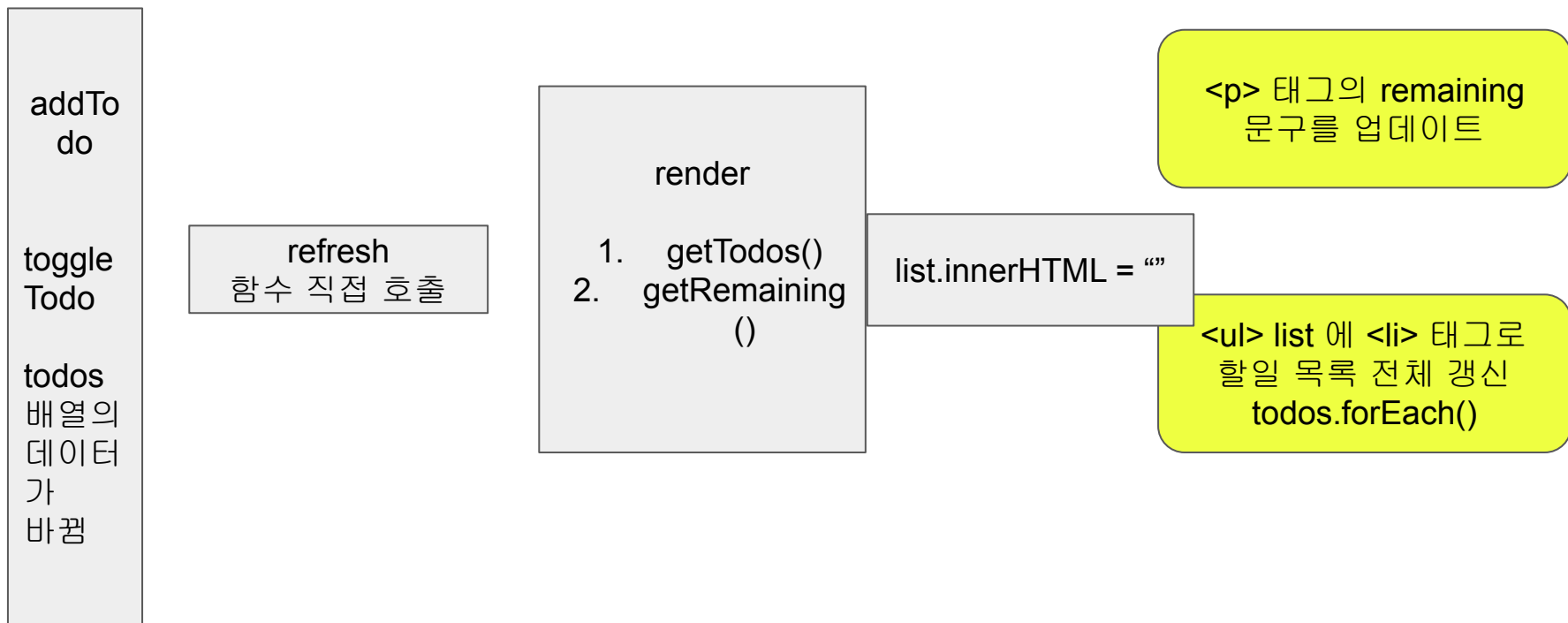


1. render 함수는 todos를 UI 그려주는 역할
2. addBtn todos (input 텍스트를) 신규 todo 배열넣고 -> render 화면을 다시그렸죠
3. 듀링고 1을 클릭하면 toggleTodo (todo 배열을 모두 돌아서 todo.id 해당되는 id의 done -> !done) -> 배열을 다시 만들 -> render 함수 이용해서 다시 그림을 그림

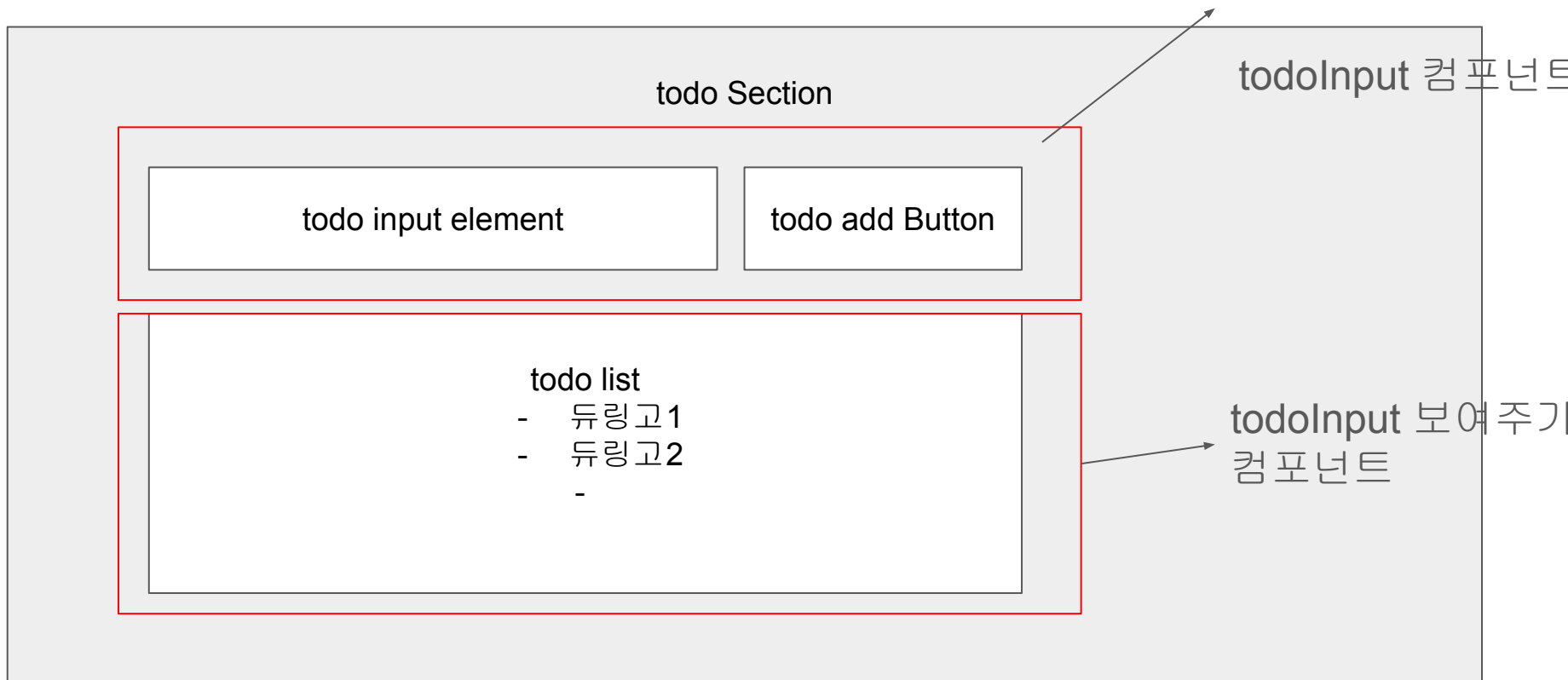
Todo Sequence diagram



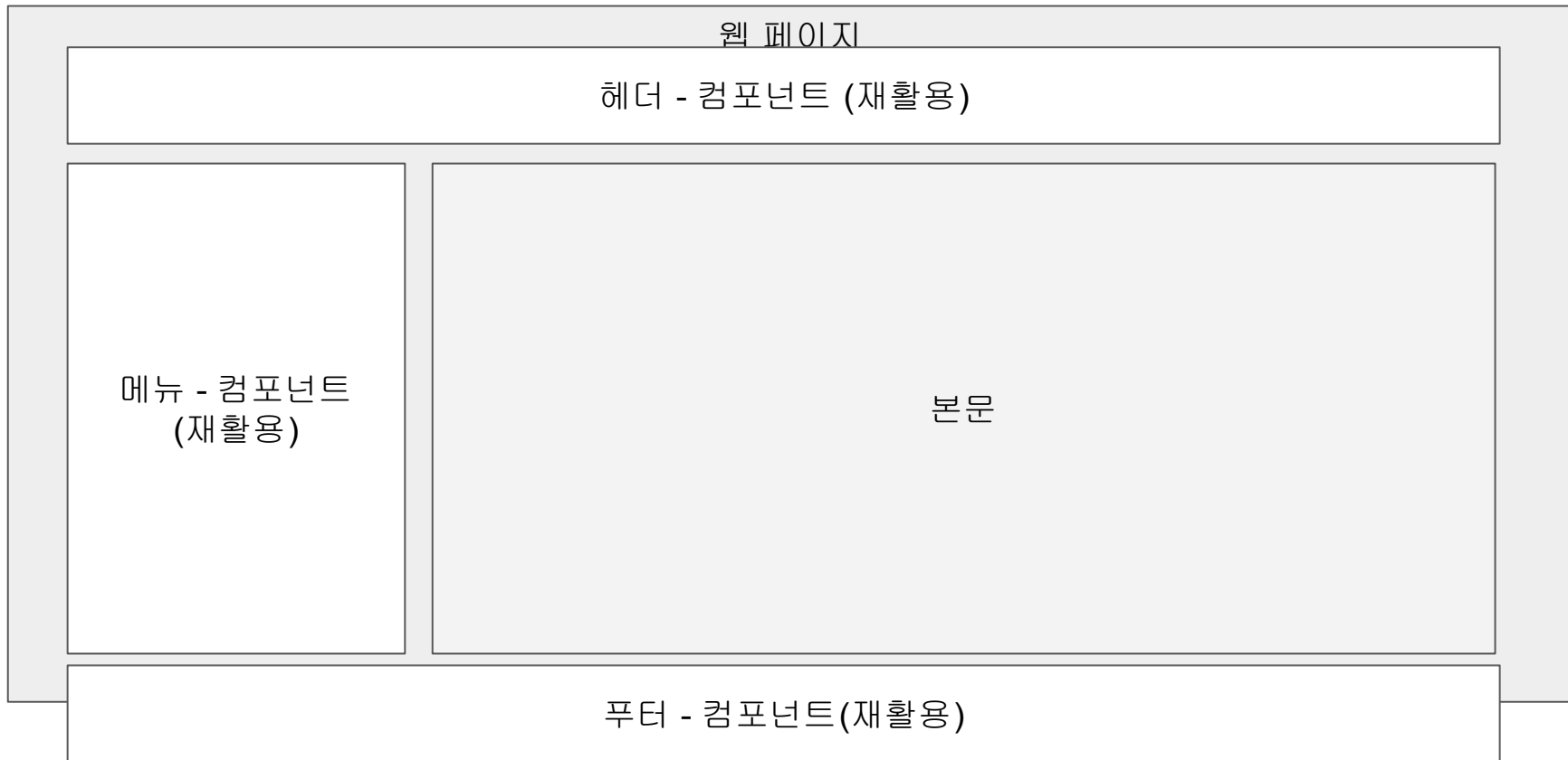
refresh 와 render



Todo



Web 페이지 구성요소



컬리N마트 넌다세일

여름 식탁을 채우는 압도적 혜택!

공지사항

비밀번호를 변경해주세요.

+more



요리의 기초

쉽고 빠른 부엌일을 위하여!

버터를 사용하기 편하게 보관하려면

아육과 근대

달래 손질하기

고사리 불리기

두부 물기 빼기



히트레시피

더 이상의 레시피는 없다!

궁채 들깨볶음

우영 감정

당근 라페 (당근채절임)

(초간단) 느타리버섯전

동남아풍 돼지갈비

키친 토크



+more

[첼시] 복숭아 오픈 샌드위치 만들어보아요.

[beantown] 사먹은 음식들이예요 - ♥

[첼시] 대전 두부두루치기 소개 드려요!

[소년공원] 미국의 졸업 시즌

[hoshidsh] 올봄 대전 탕방기+양상추 볶음밥

[고독은 나의 힘] 약속했던 파이 사진들

[행복나눔미소] 196차 봉사후기) 2026년 5월 불낙전골, 낙...

[juju] 오랜만에 왔어요

[하얀쌀밥] 아침은먹었나요?

[점점] 마늘쭀퍼스타

[첼시] 딸보이 맛있게 먹는 법 2 사진

[첼시] 제가 떡볶이 맛있게 먹는법

리빙 데코



+more

[wooo] 마지막 인사 드립니다.

[탕낭자] 크리스마스 파티 풍선으로 장식해봤어요

[wooo] 은퇴, 그리고 인형만들기

[wooo] 프랑스자수와 인형 액자만들기

[wooo] 뜨개놀이와 가방, 그리고 새로운 놀이

[얼렁뚱땅] 자투리천을 활용한 가방 만들기

[wooo] 나의 외도... 뜨개

[Juliana7] 풍납뜨개 도깨비 모입니다.^^

아이디

아이디

로그인

비밀번호

비밀번호

[회원가입] 아이디/비밀번호 찾기

Chrome으로 전환하세요



Chrome을 사용해서
작업을 더 빠르게 처리할 수 있는
3가지 방법

- 1. 새 탭 (Ctrl+N) 새 탭으로 페이지를 열기
- 2. 새 창 (Ctrl+N) 새 창으로 페이지를 열기
- 3. 새 인스턴스 (Ctrl+N) 새 인스턴스로 페이지를 열기



다운로드하기



+ 최근 많이 읽은 글

66세인 최화정이..

장기하랑 같은 나이 45살 사...

장원영이랑 뽀 사관다는 루머 ...

이재명 정진 놓은건가요?

삼전 닉스 오늘...

와 역시 최민식

반도체 오늘 왜 이래요????

17억 3200만원이래요 검찰개...

안전한도 대실랑

박스를 그냥 버리다니...

JSX 가 변환

App.jsx
return (<div> .. </div>)

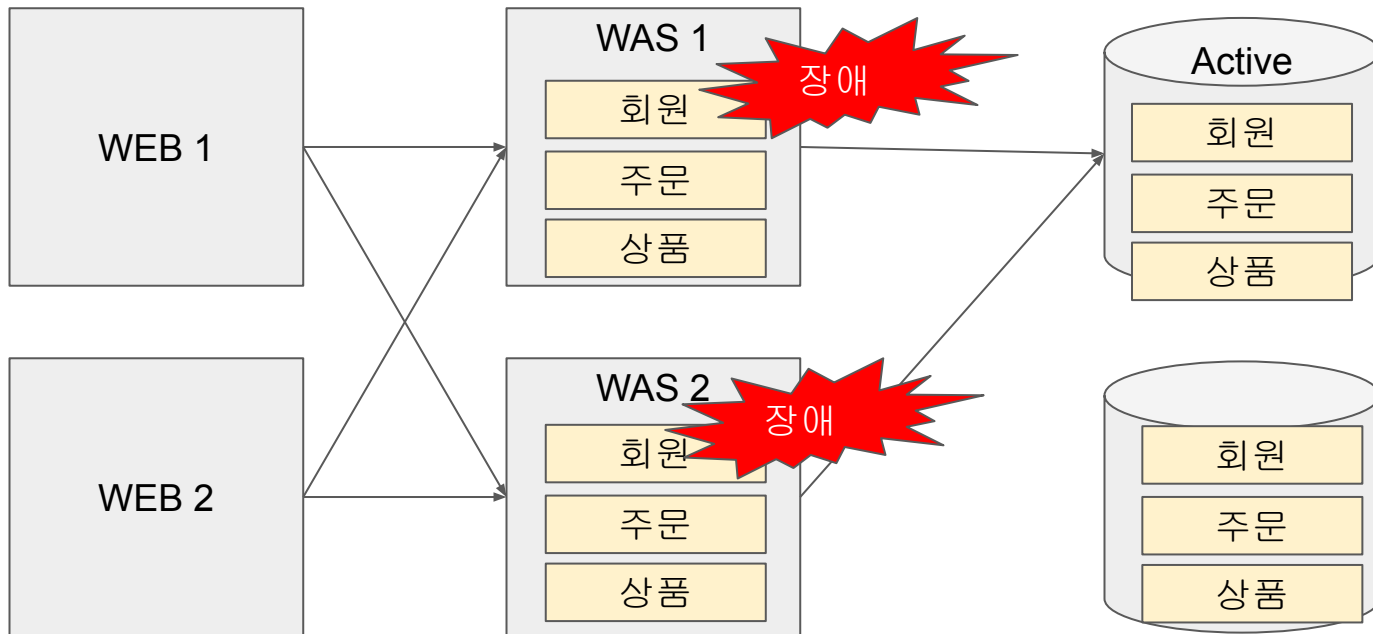
@vitejs/plugin-react

일반 JS - dist/
React.createElement
일반 자바스크립트로
변환

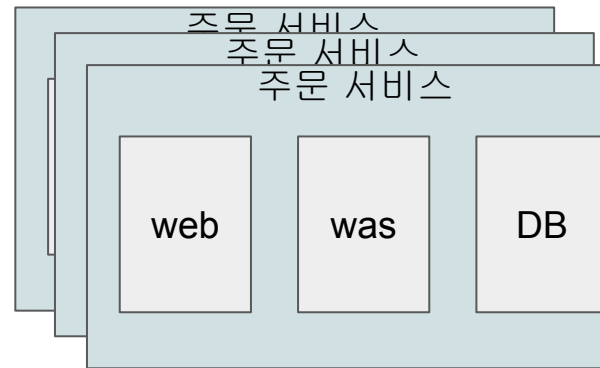
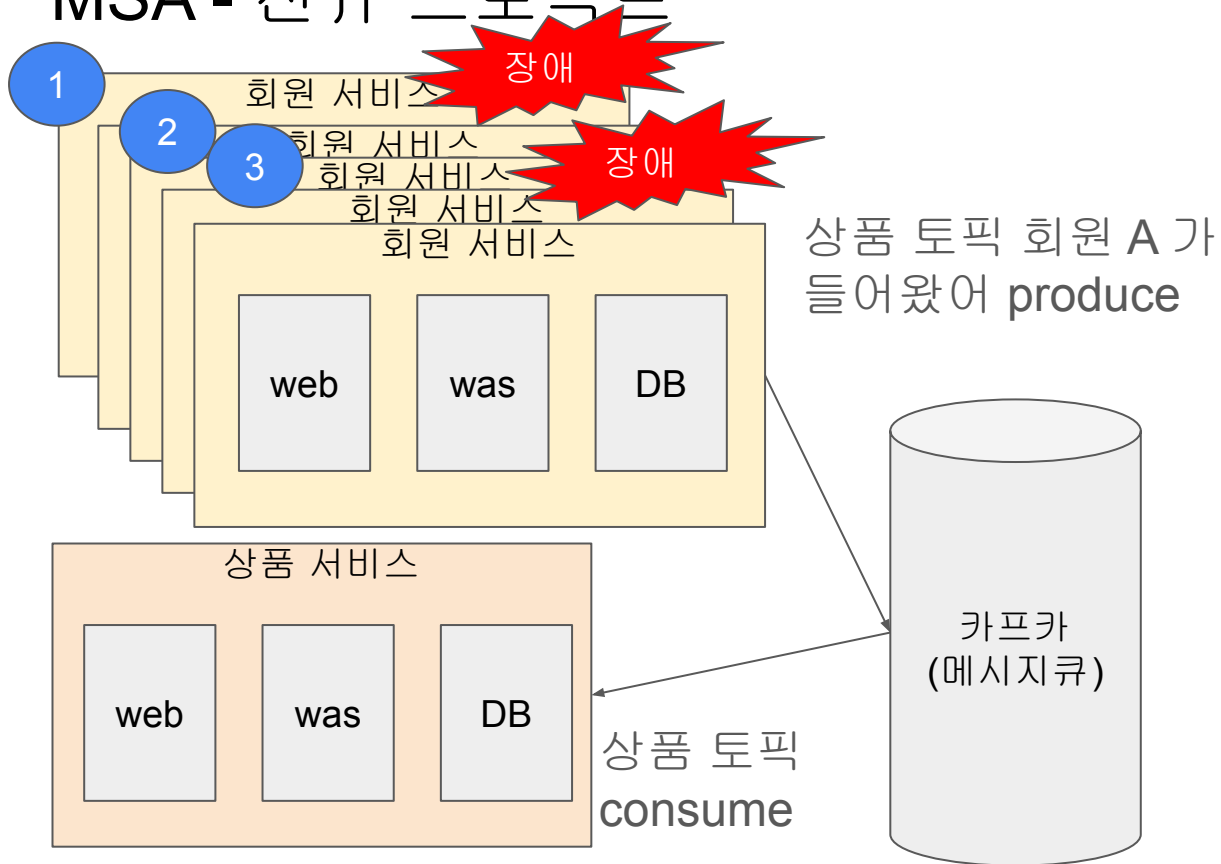
브라우저 화면에서
내용을 볼 수 있었죠

npm run dev, npm run build

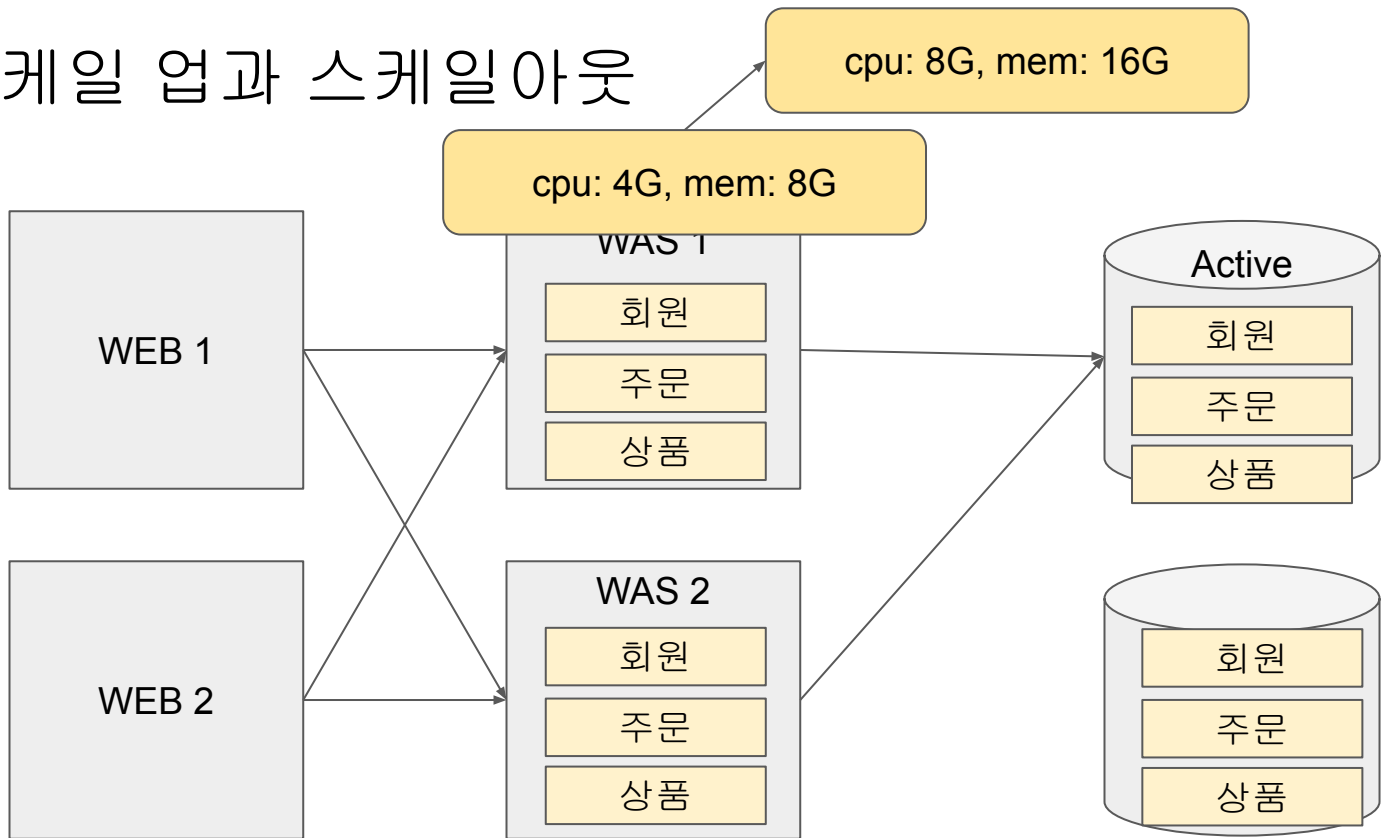
MSA(Micro Service Architecture)



MSA - 신규 프로젝트



스케일 업과 스케일아웃



JSX 기본 규칙

App.jsx

```
return (  
  <p> 안녕, {name} 님 </p>  
)
```

<p> : JSX 태그

name : JS 변수(값이 치환)

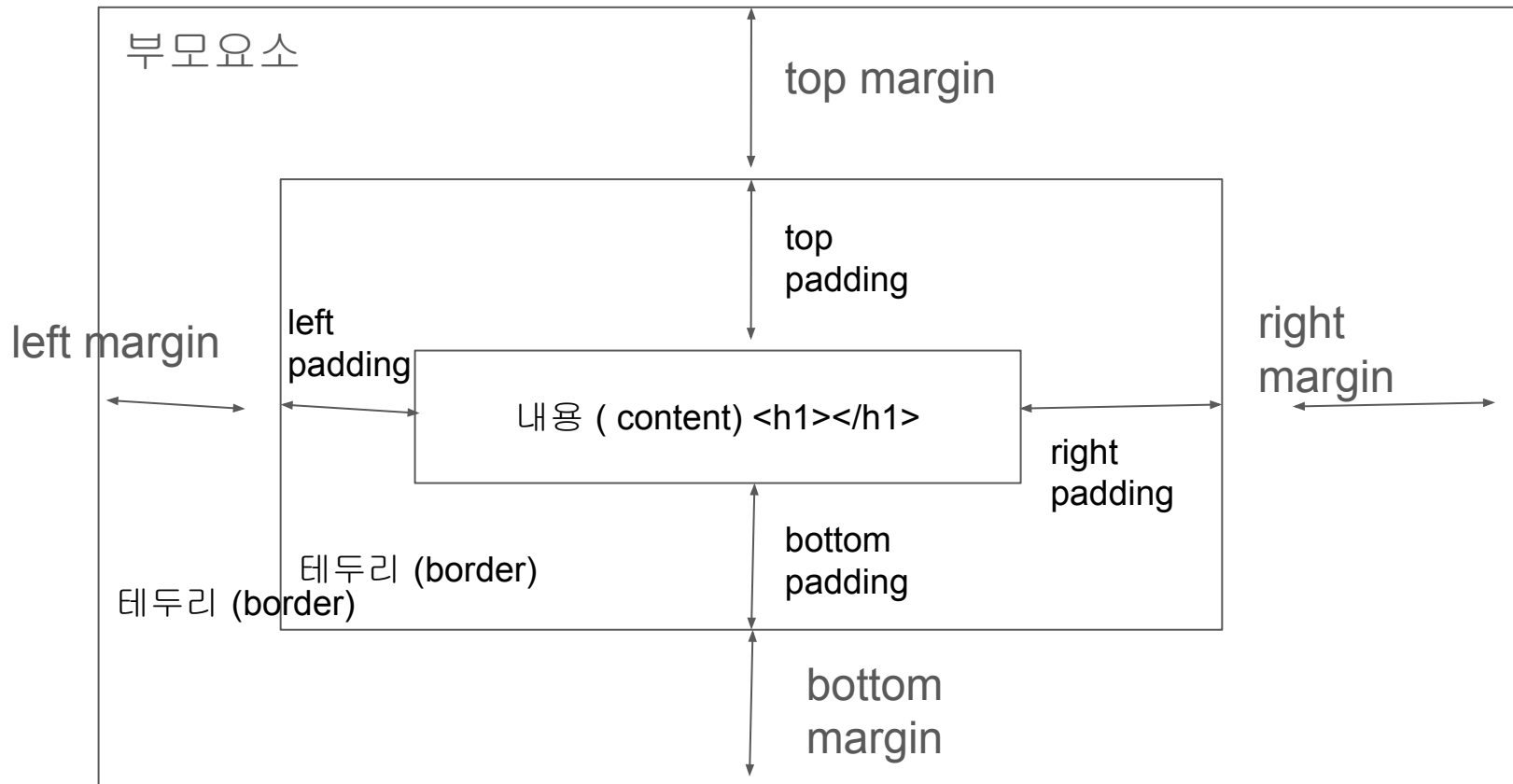
</p> : 반드시 태그는 닫아야함

class="app" -> className="app"

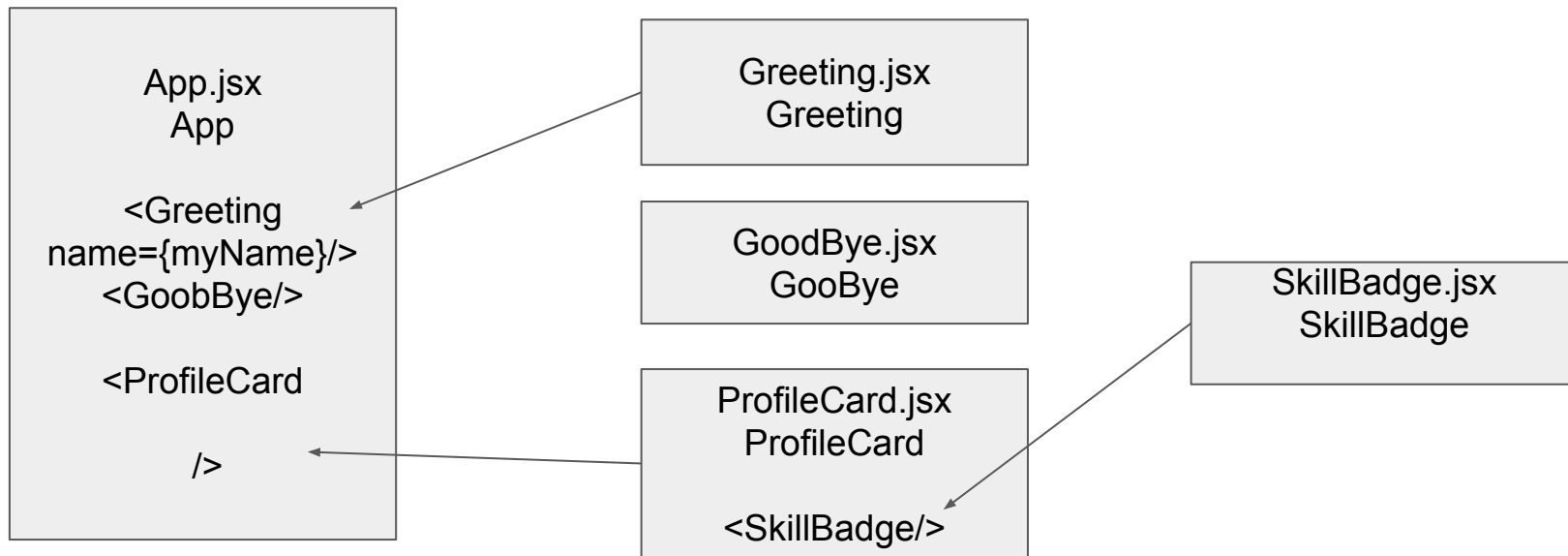
name="myName" name={myName}

return (<div></div><p></p>) -> return
(<></>) - 하나의 엘리먼트로만 리턴
해야함

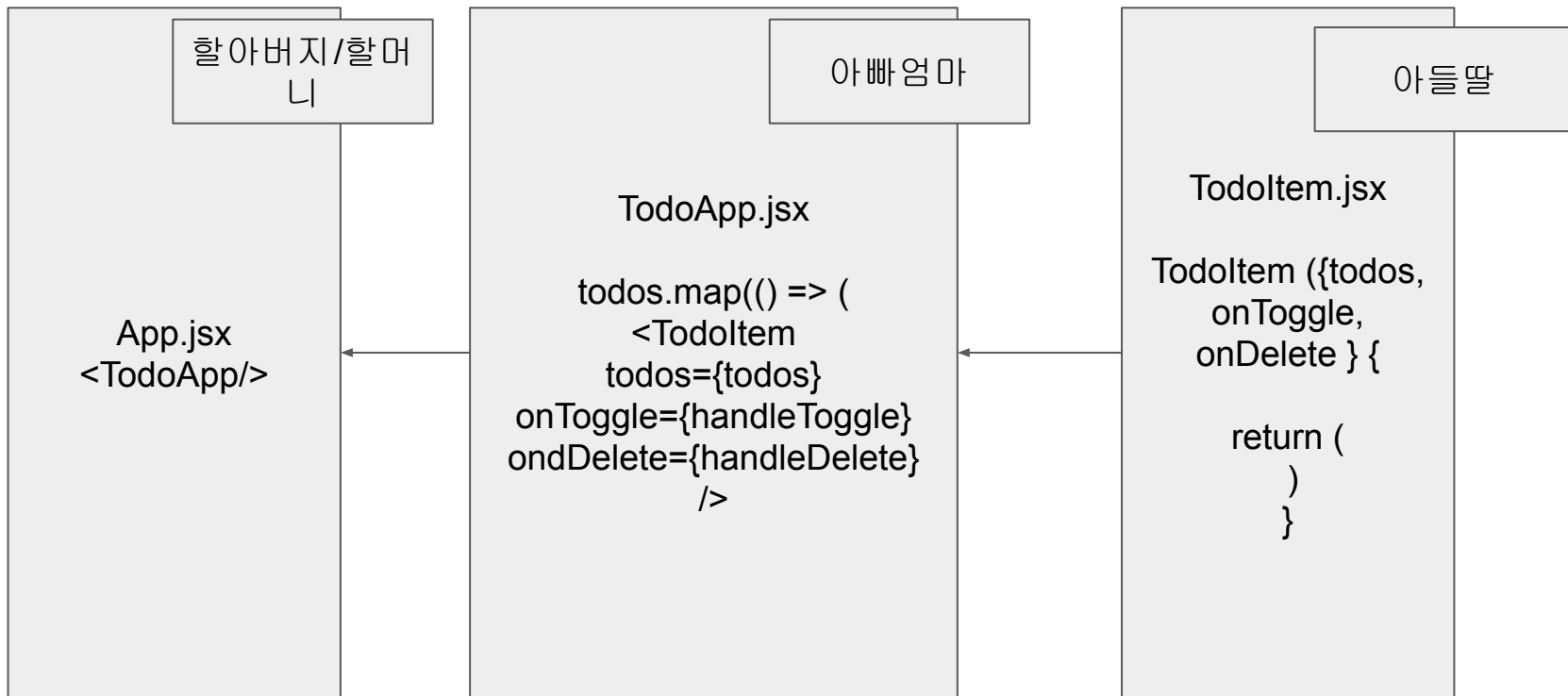
박스 모델



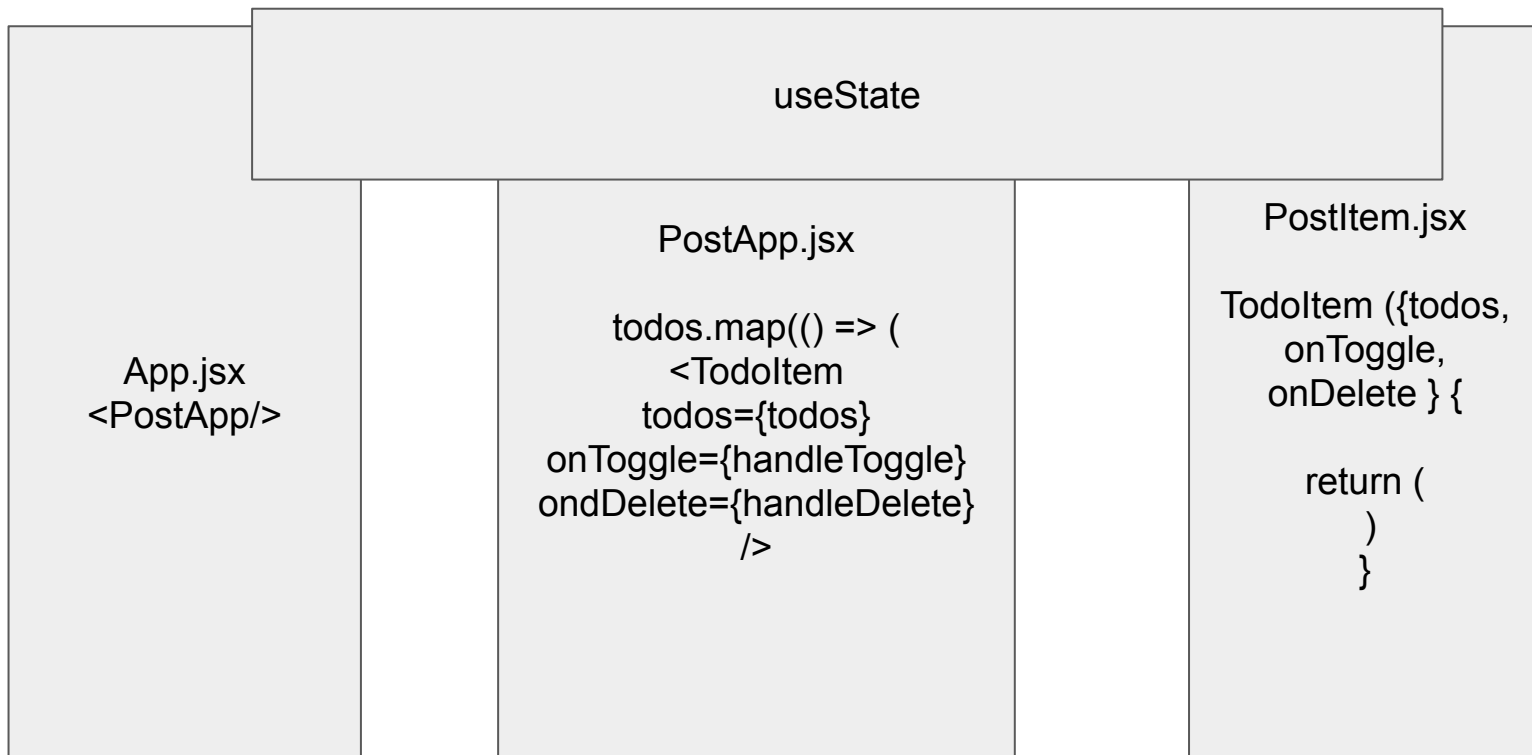
React Component



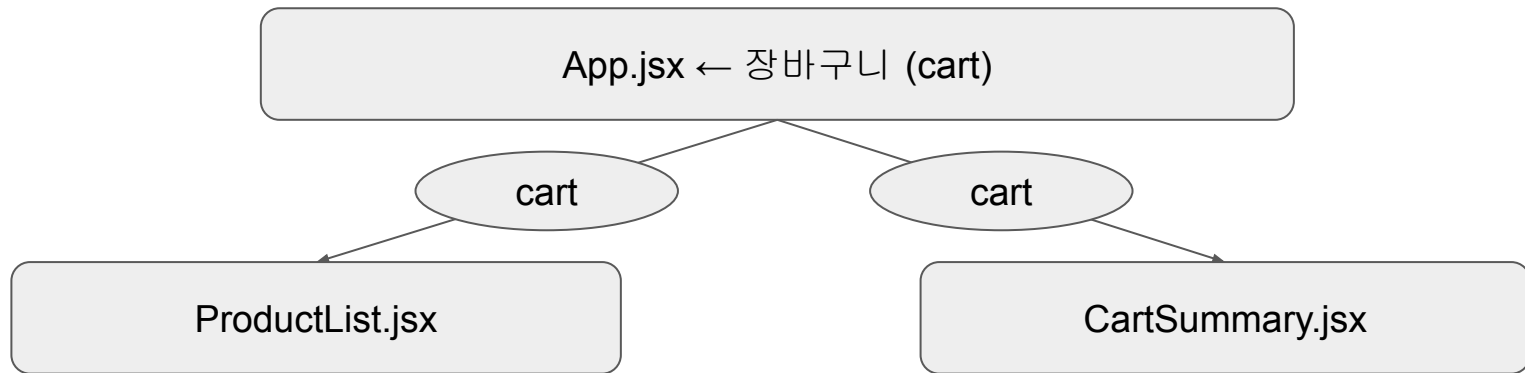
TodoApp



PostApp

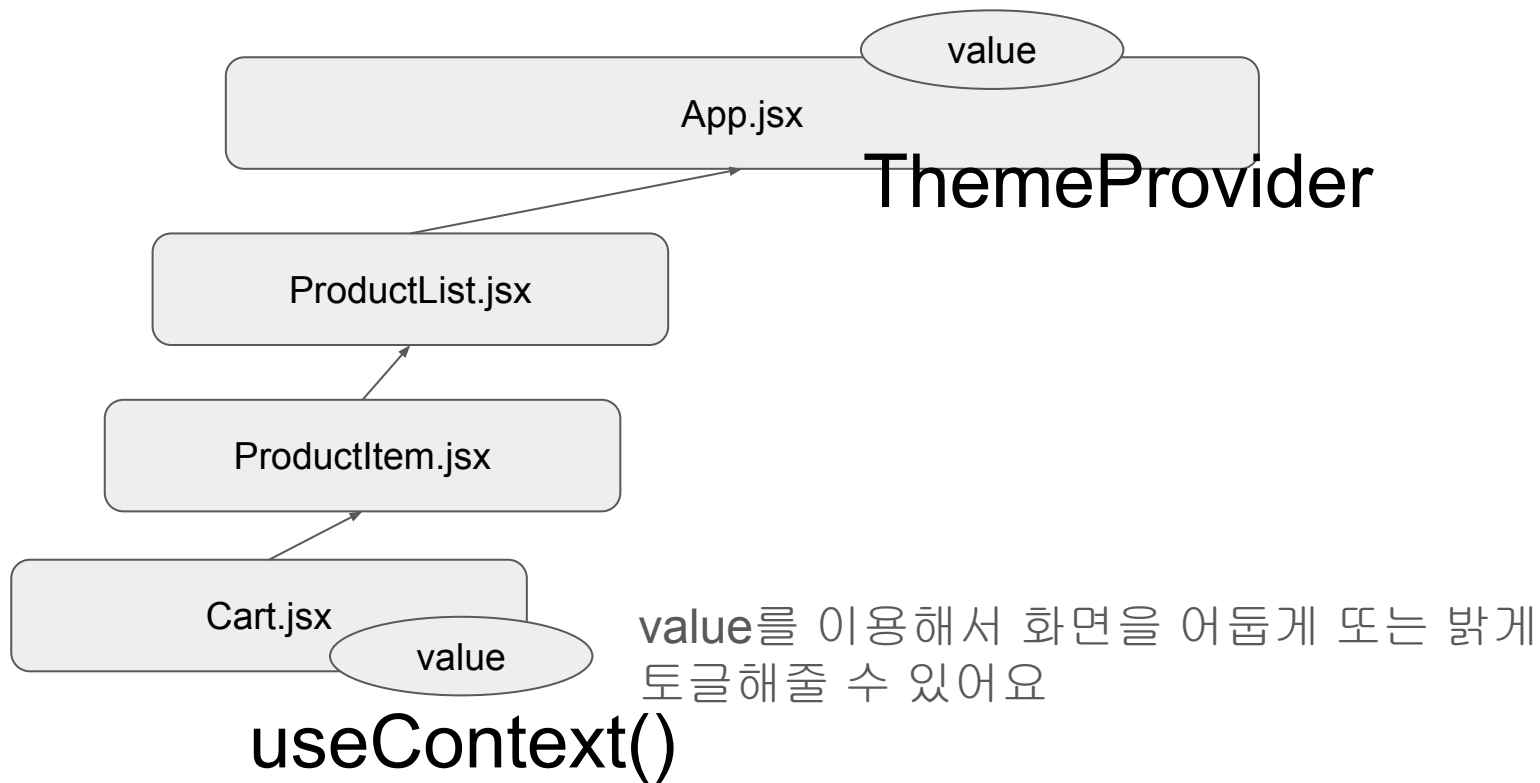


공유되는 state



ProductList, CartSummary를 형제라고 한다면 형제는 부모를 통해서만 연결

useContext



Context

ThemeProvider : 사물함 관리자

ThemeContext : 사물함 자체

value : 사물함 안의 물건

useTheme() : 사물함을 사용하기 위한 열쇠

props drilling

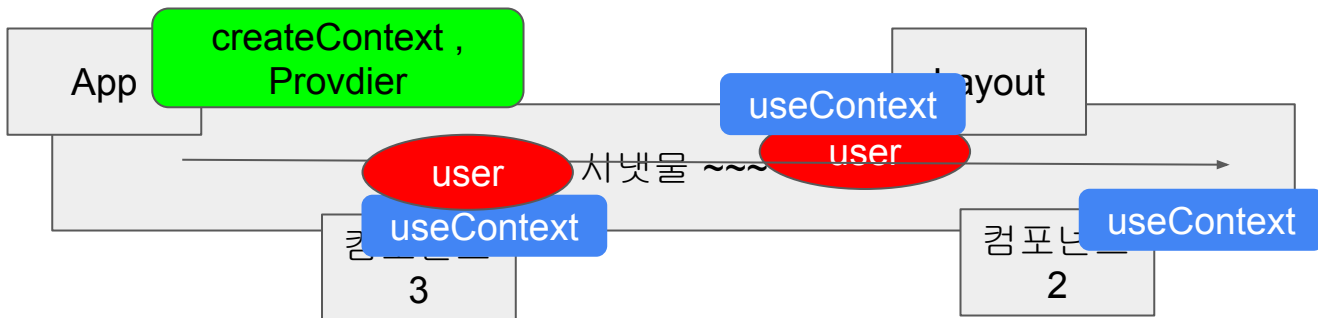
```
function App() {  
  const user = { name: "지훈" }  
  return <Layout user={user} /> //생성  
}  
  
function Layout ({user}) {  
  return <Header user={user}/> //전달  
}
```

```
function Header({user}) {  
  return <Profile user={user} />  
  //전달  
}  
  
function Profile ({user}) {  
  return <span>{user.name}</span>  
  //실제  
}
```

Context == wifi 공유기

값을 공중에 뿌려두면, 필요한 컴포넌트가 어디서든 직접 잡아서 쓴다.
중간컴포넌트는 배달할 필요 없다.

- 1) createContext : 값이 흐를 채널을 하나 만든다.
- 2) Provider : 그 채널에 값을 실어 뿌린다.
- 3) useContext: 필요한 컴포넌트가 값을 잡는다.



Context로 고친 코드

```
const UserContext= createContext(null) // 1. 채널을 만들어 -> 시내물을 만들
```

```
function App() {
```

```
    const user = {name : "지훈"}
```

```
    return <UserContext.Provider value={user}><Layout/>  
</UserContext.Provider>
```

```
    // 2. user 값을 하위로 뿌려
```

```
}
```

```
function Layout() {return <Header/>}// 깨끗
```

```
function Header() {return <Profile/>}// 깨끗
```

```
function Profile() {
```

```
    const user = useContext(UserContext); // 3. 잡아 값을 시냇물에서
```

```
    return <span> {user.name} </span>
```

```
}// 깨끗
```

1. **Layout - Header** 에서 **user** 가 사라짐
2. **Profile** 에서 필요한 값을 직접 꺼냄
3. 중간단계 아무리 깊어도 **100개**까지 간다고해도 배달코드는 0 줄
4. **props** 로 넘기던걸 모두 없앴어요